

Knowledge Compilation: Theory, Practice, and Applications

Jean-Marie Lagniez¹ Johannes Fichte²

¹CRIL, Univ. Artois & CNRS, France

²Linköping University, Sweden

Outline

- 1 How to model using SAT
- 2 Knowledge Compilation
- 3 d-DNNF Queries
- 4 Conclusion and References

How to model using SAT

Outline

1 How to model using SAT

■ SAT

- Definition
- Complexity
- Modeling

■ #SAT

■ $\exists$ #SAT

■ Weighted #SAT

2 Knowledge Compilation

3 d-DNNF Queries

4 Conclusion and References

Outline

1 How to model using SAT

■ SAT

■ Definition

- Complexity
- Modeling

■ #SAT

■ $\exists$ #SAT

■ Weighted #SAT

2 Knowledge Compilation

3 d-DNNF Queries

4 Conclusion and References

The SAT problem

	$a \vee \neg b$	$\neg a \vee c$	
$\neg a \vee b$	$\neg b \vee c \vee d$	$\neg b \vee c \vee \neg d$	$\neg b \vee c$
	$e \vee f$	$e \vee \neg f$	

- Propositional variables: a, b, c, d, e, f
- Literals: $a, \neg a$
- Clauses: $a \vee b$
- A CNF formula F : $(a \vee \neg b) \wedge (\neg a \vee c) \wedge \dots$

The SAT problem

$\neg a \vee b$	$a \vee \neg b$	$\neg a \vee c$							
	$\neg b \vee c \vee d$	$\neg b \vee c \vee \neg d$	$\neg b \vee c$						
	$e \vee f$	$e \vee \neg f$							
<table><tr><td>a</td><td>b</td><td>c</td><td>d</td><td>e</td><td>f</td></tr></table>				a	b	c	d	e	f
a	b	c	d	e	f				

- Propositional variables: a, b, c, d, e, f
- Literals: $a, \neg a$
- Clauses: $a \vee b$
- A CNF formula F : $(a \vee \neg b) \wedge (\neg a \vee c) \wedge \dots$
- An **assignment** is a mapping $\mathcal{I} : \text{var}(F) \rightarrow \{\perp, \top\}$ and
 $\mathcal{I}$ is **satisfying** if results in F being $\top$ (true)

The SAT problem

	$a \vee \neg b$	$\neg a \vee c$							
$\neg a \vee b$	$\neg b \vee c \vee d$	$\neg b \vee c \vee \neg d$	$\neg b \vee c$						
	$e \vee f$	$e \vee \neg f$							
<table><tr><td>a</td><td>b</td><td>c</td><td>d</td><td>e</td><td>f</td></tr></table>				a	b	c	d	e	f
a	b	c	d	e	f				

- Propositional variables: a, b, c, d, e, f
- Literals: $a, \neg a$
- Clauses: $a \vee b$
- A CNF formula F : $(a \vee \neg b) \wedge (\neg a \vee c) \wedge \dots$
- An **assignment** is a mapping $\mathcal{I} : \text{var}(F) \rightarrow \{\perp, \top\}$ and
 $\mathcal{I}$ is **satisfying** if results in F being $\top$ (true)
- SAT problem: does there exist a satisfying assignment for F ?

The SAT problem

$\neg a \vee b$	$a \vee \neg b$	$\neg a \vee c$			
	$\neg b \vee c \vee d$	$\neg b \vee c \vee \neg d$	$\neg b \vee c$		
	$e \vee f$	$e \vee \neg f$			
a	b	c	d	e	f
$\perp$	T	T	T	T	$\perp$

- Propositional variables: a, b, c, d, e, f
- Literals: $a, \neg a$
- Clauses: $a \vee b$
- A CNF formula F : $(a \vee \neg b) \wedge (\neg a \vee c) \wedge \dots$
- An **assignment** is a mapping $\mathcal{I} : \text{var}(F) \rightarrow \{\perp, T\}$ and
 $\mathcal{I}$ is **satisfying** if results in F being T (true)
- SAT problem: does there exist a satisfying assignment for F ?

The SAT problem

$$\neg a \vee b \qquad a \vee \neg b \qquad \neg a \vee c \qquad \neg b \vee c$$
$$\neg b \vee c \vee d \qquad \neg b \vee c \vee \neg d$$
$$e \vee f \qquad e \vee \neg f$$

a	b	c	d	e	f
$\perp$	$\perp$	T	T	T	$\perp$

- Propositional variables: a, b, c, d, e, f
- Literals: $a, \neg a$
- Clauses: $a \vee b$
- A CNF formula F : $(a \vee \neg b) \wedge (\neg a \vee c) \wedge \dots$
- An **assignment** is a mapping $\mathcal{I} : \text{var}(F) \rightarrow \{\perp, \text{T}\}$ and $\mathcal{I}$ is **satisfying** if results in F being T (true)
- SAT problem: does there exist a satisfying assignment for F ?

Outline

1 How to model using SAT

■ SAT

- Definition
- Complexity
- Modeling

■ #SAT

■ $\exists$ #SAT

■ Weighted #SAT

2 Knowledge Compilation

3 d-DNNF Queries

4 Conclusion and References

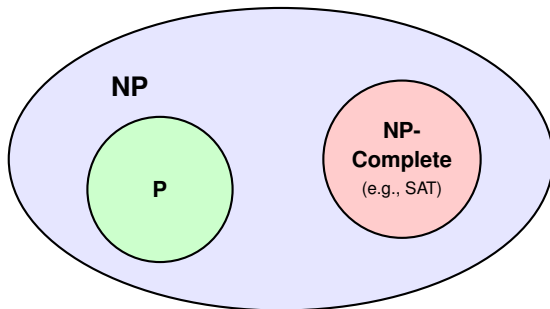
A Brief Detour into Complexity Theory

- To understand why using SAT, we must understand how hard SAT is to solve.
- **Class P (Polynomial Time):**
 - Problems that can be **solved** efficiently by a deterministic algorithm.
 - Examples: Sorting an array, finding the shortest path in a graph.
- **Class NP (Nondeterministic Polynomial Time):**
 - Problems where a proposed solution can be **verified** efficiently.
 - Example: Sudoku. Hard to solve from scratch, but easy to check if a completed grid is correct.

Does $P = NP$? (2nd Millennium Prize Problem of Mathematics)

SAT and NP-Completeness

- **Cook-Levin Theorem (Cook, 1971):** SAT was the first problem proven to be **NP-Complete**.
- What does NP-Complete mean?
 - It is in NP (a satisfying assignment is easy to check).
 - It is as hard as the hardest problems in NP.
 - *Every problem in NP can be translated (reduced) into a SAT problem efficiently.*
- **ETH (Exponential-Time Hypothesis):** Worst-case SAT cannot be solved significantly better than exponential time $\mathcal{O}(2^n)$ (more precisely: there is no subexponential-time algorithm for 3-SAT).



The SAT Solving Paradox

Theory vs. Practice

- **Theory:** SAT is NP-Complete $\Rightarrow$ intractable in the worst case.
- **Practice:** Modern SAT solvers can routinely solve industrial instances with **millions of variables and clauses**.
- **How is this possible?**
 - Real-world problems (like hardware verification or XAI) are highly structured, unlike random formulas.
 - Modern solvers exploit this hidden structure.
- **The CDCL (Conflict-Driven Clause Learning) Revolution (Silva and Sakallah, 1996):**
 - When the solver makes a mistake, it analyzes the logical conflict.
 - It *learns* a new clause to prevent making the exact same mistake again.
 - It then backtracks non-chronologically to jump out of dead search spaces.

Outline

1 How to model using SAT

■ SAT

- Definition
- Complexity
- Modeling

■ #SAT

■ $\exists$ #SAT

■ Weighted #SAT

2 Knowledge Compilation

3 d-DNNF Queries

4 Conclusion and References

Graph Coloring Problem

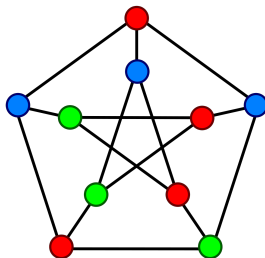
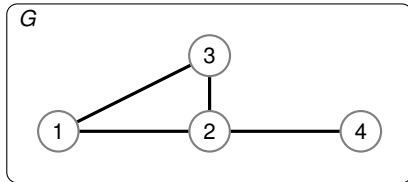


Figure: $\chi(G) = 3$

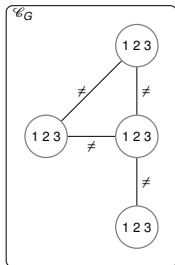
- To assign a minimum number of colors to all vertices s.t. no adjacent vertices have the same color
- The smallest number of colors: *chromatic number* ($\chi(G)$)
- Determining the chromatic number of a graph is an NP-hard task

Classical Graph Coloring Encoding

- *Decision Problem:* Can we color $G = (V, E)$ with 3 colors?



- We can express the problem with a set of constraints

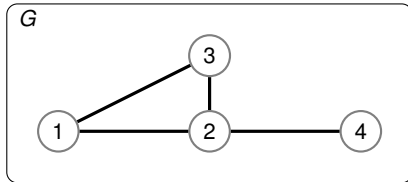


- 1 x_{ij} is true if the vertex i is colored with the color j
- 2 At least one color by vertex: $\bigwedge_{i=1}^4 (\bigvee_{j=1}^3 x_{ij})$
- 3 At most one color by vertex:
 $\bigwedge_{i=1}^4 \bigwedge_{j=1}^3 \bigvee_{k=j+1}^3 (\neg x_{ij} \vee \neg x_{ik})$
- 4 Two adjacency vertices cannot share the same color:
 $\forall \{i, j\} \in E, \bigwedge_{k=1}^3 \neg x_{ik} \vee \neg x_{jk}$

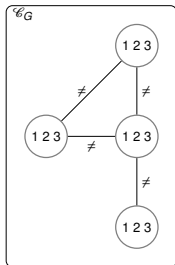
Live example?

Classical Graph Coloring Encoding

- *Decision Problem:* Can we color $G = (V, E)$ with 3 colors?



- We can express the problem with a set of constraints

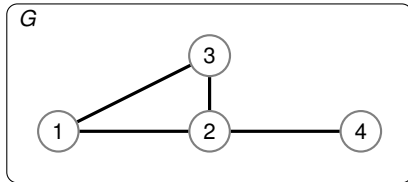


- 1 x_{ij} is true if the vertex i is colored with the color j
- 2 At least one color by vertex: $\bigwedge_{i=1}^4 (\bigvee_{j=1}^3 x_{ij})$
- 3 At most one color by vertex:
 $\bigwedge_{i=1}^4 \bigwedge_{j=1}^3 \bigvee_{k=j+1}^3 (\neg x_{ij} \vee \neg x_{ik})$
- 4 Two adjacency vertices cannot share the same color:
 $\forall \{i, j\} \in E, \bigwedge_{k=1}^3 \neg x_{ik} \vee \neg x_{jk}$

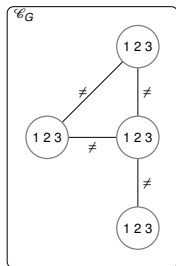
Live example?

Classical Graph Coloring Encoding

- *Decision Problem:* Can we color $G = (V, E)$ with 3 colors?



- We can express the problem with a set of constraints



- 1 x_{ij} is true if the vertex i is colored with the color j
- 2 At least one color by vertex: $\bigwedge_{i=1}^4 (\bigvee_{j=1}^3 x_{ij})$
- 3 At most one color by vertex:
 $\bigwedge_{i=1}^4 \bigwedge_{j=1}^3 \bigvee_{k=j+1}^3 (\neg x_{ij} \vee \neg x_{ik})$
- 4 Two adjacency vertices cannot share the same color:
 $\forall \{i, j\} \in E, \bigwedge_{k=1}^3 \neg x_{ik} \vee \neg x_{jk}$

Live example?

The Counting Challenge

Finding one coloring is “easy” (SAT). But how many colorings exist?

Outline

1 How to model using SAT

■ SAT

■ #SAT

- Definition
- Complexity
- Modeling

■ $\exists\#\text{SAT}$

■ Weighted #SAT

2 Knowledge Compilation

3 d-DNNF Queries

4 Conclusion and References

Outline

1 How to model using SAT

- SAT

- #SAT**

 - Definition

 - Complexity

 - Modeling

- $\exists\#\text{SAT}$

- Weighted #SAT

2 Knowledge Compilation

3 d-DNNF Queries

4 Conclusion and References

The Model Counting Problem (#SAT)

Definition

Model Counting is the problem of computing the number of total satisfying assignments (models) for a given propositional formula Σ .

$$\Sigma \mapsto \|\Sigma\| = |\{\mathcal{I} \mid \mathcal{I} \models \Sigma\}|$$

Example: Let $\Sigma = (\neg a \vee \neg b \vee \neg c) \wedge (a \vee c) \wedge (a \vee b) \wedge (\neg b \vee \neg c)$

The models of Σ over variables $\{a, b, c\}$ are:

a	b	c
T	$\perp$	$\perp$
T	$\perp$	T
T	T	$\perp$

Answer: $\|\Sigma\| = 3$

Outline

1 How to model using SAT

- SAT

- #SAT**

 - Definition

 - Complexity**

 - Modeling

- $\exists\#\text{SAT}$

- Weighted #SAT

2 Knowledge Compilation

3 d-DNNF Queries

4 Conclusion and References

The Complexity of Model Counting

■ A Central Problem in AI

Model counting is crucial for various applications:

- Probabilistic reasoning (Bayesian inference)
- Automated verification
- Planning under uncertainty

■ Computational Hardness

#SAT is **#P-complete** (Valiant, 1979).

- It is significantly harder than SAT (NP-complete).

The Power of Counting: Toda's Theorem 1991

Seinosuke Toda (Gödel Prize 1998)

The entire Polynomial Hierarchy is contained in $P^{\#P}$:

$$PH \subseteq P^{\#P}$$

Outline

1 How to model using SAT

- SAT

- #SAT

 - Definition

 - Complexity

 - Modeling

- $\exists\#\text{SAT}$

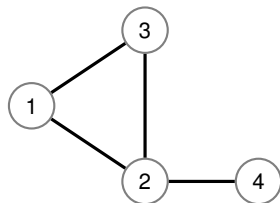
- Weighted #SAT

2 Knowledge Compilation

3 d-DNNF Queries

4 Conclusion and References

The SAT Encoding in Practice



$G = (V, E)$

CNF Formula Σ_G ($n = 4, k = 3$) where $X = \text{var}(\Sigma_G)$

1. At least one color per vertex:

$$(x_{1,1} \vee x_{1,2} \vee x_{1,3}) \wedge (x_{2,1} \vee x_{2,2} \vee x_{2,3}) \wedge (x_{3,1} \vee x_{3,2} \vee x_{3,3}) \wedge (x_{4,1} \vee x_{4,2} \vee x_{4,3})$$

2. At most one color per vertex (Mutual Exclusion):

$$\begin{aligned} &(\neg x_{1,1} \vee \neg x_{1,2}) \wedge (\neg x_{1,1} \vee \neg x_{1,3}) \wedge (\neg x_{1,2} \vee \neg x_{1,3}) \\ &(\neg x_{2,1} \vee \neg x_{2,2}) \wedge (\neg x_{2,1} \vee \neg x_{2,3}) \wedge (\neg x_{2,2} \vee \neg x_{2,3}) \\ &(\neg x_{3,1} \vee \neg x_{3,2}) \wedge (\neg x_{3,1} \vee \neg x_{3,3}) \wedge (\neg x_{3,2} \vee \neg x_{3,3}) \\ &(\neg x_{4,1} \vee \neg x_{4,2}) \wedge (\neg x_{4,1} \vee \neg x_{4,3}) \wedge (\neg x_{4,2} \vee \neg x_{4,3}) \end{aligned}$$

3. Adjacency (Edge Constraints):

$$\begin{aligned} &(\neg x_{1,1} \vee \neg x_{2,1}) \wedge (\neg x_{1,2} \vee \neg x_{2,2}) \wedge (\neg x_{1,3} \vee \neg x_{2,3}) && \text{(Edge 1-2)} \\ &(\neg x_{1,1} \vee \neg x_{3,1}) \wedge (\neg x_{1,2} \vee \neg x_{3,2}) \wedge (\neg x_{1,3} \vee \neg x_{3,3}) && \text{(Edge 1-3)} \\ &(\neg x_{2,1} \vee \neg x_{3,1}) \wedge (\neg x_{2,2} \vee \neg x_{3,2}) \wedge (\neg x_{2,3} \vee \neg x_{3,3}) && \text{(Edge 2-3)} \\ &(\neg x_{2,1} \vee \neg x_{4,1}) \wedge (\neg x_{2,2} \vee \neg x_{4,2}) \wedge (\neg x_{2,3} \vee \neg x_{4,3}) && \text{(Edge 2-4)} \end{aligned}$$

- **Variables:** $4 \times 3 = 12$ Boolean variables.
- **Clauses:** $4 + (4 \times 3) + (4 \times 3) = 28$ clauses.
- **The Goal:** Compute $\|\Sigma_G\| = |\{\mathcal{J} \in \{0, 1\}^X \mid \mathcal{J} \models \Sigma_G\}|$.

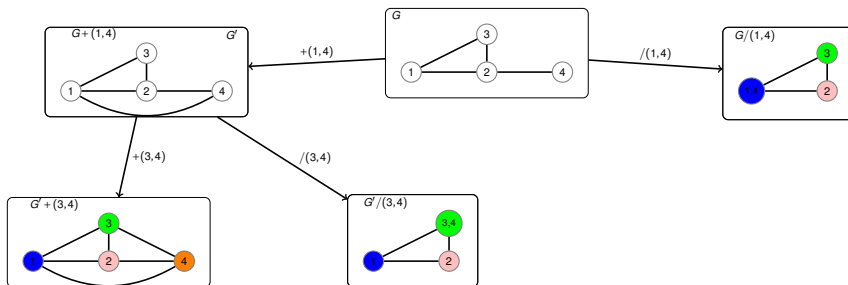
Even for this small graph, a manual count is tedious. Knowledge Compilation automates this by transforming Σ_G into a tractable form.

Zykov's deletion-contraction recurrence

Operation on the graph:

- We branch on the missing edges;
- $G+(i,j)$ means G with the edge (i,j) ;
- $G/(i,j)$ means we merge the vertices i and j in G .

Zykov's recurrence: $\chi(G) = \min\{ \chi(G/(ij)), \chi(G+(ij)) \} \forall (i,j) \notin E$



Encoding Zykov's Graph Coloring in CNF (Glorian et al., 2019)

1. The Equivalence Relation (s_{ij})

$s_{ij} \equiv$ vertices i and j share the same color.

- **Edge Constraints:** $\neg s_{ij} \quad \forall (i, j) \in E$
- **Transitivity:** $(\neg s_{ij} \vee \neg s_{jk} \vee s_{ik}) \quad \forall i, j, k \in V$

2. The k -Color Constraint

Identify “color leaders” c_j who start a new color:

$$c_j \iff \bigwedge_{i < j} \neg s_{ij} \quad \text{and ensure} \quad \sum_{j=1}^n c_j \leq k$$

The Compilation Challenge

Encoding $\sum c_j \leq k$ into CNF requires **auxiliary variables** (e.g., via Totalizers or Adder circuits).

- We only care about the models over the original s_{ij} and c_j variables.
- **Problem:** Standard counting over the compiled circuit will be biased by these extra bits.

Outline

1 How to model using SAT

- SAT

- #SAT

- $\exists\#\text{SAT}$

 - Definition

 - Modeling

- Weighted #SAT

2 Knowledge Compilation

3 d-DNNF Queries

4 Conclusion and References

Outline

1 How to model using SAT

- SAT
- #SAT
- $\exists\#\text{SAT}$
 - Definition
 - Modeling
- Weighted #SAT

2 Knowledge Compilation

3 d-DNNF Queries

4 Conclusion and References

Projected Model Counting ($\exists\#\text{SAT}$)

The Problem: Auxiliary Variables

Modern SAT encodings Σ often use a set of variables $\mathbf{X} = \mathbf{P} \cup \mathbf{A}$:

- **P: Primary variables** (e.g., the actual colorings x_{ik}).
- **A: Auxiliary variables** (e.g., Tseytin variables, cardinality counters).

Definition (Projected Model Count)

Given a formula Σ over $\mathbf{P} \cup \mathbf{A}$, the projected model count over $\mathbf{P}$ is the number of assignments to $\mathbf{P}$ that can be extended to a model of Σ :

$$\|\Sigma\|_{\mathbf{P}} = |\{\mathcal{I}_{\mathbf{P}} \mid \exists \mathcal{I}_{\mathbf{A}} : (\mathcal{I}_{\mathbf{P}} \cup \mathcal{I}_{\mathbf{A}}) \models \Sigma\}|$$

Standard #SAT

Counts every bit-string

$$|\text{Models}| = 2^{|\mathbf{A}|} \times |\text{Solutions}|$$

(if $\mathbf{A}$ is free/unconstrained)

$\Rightarrow$

Projected #SAT

Counts unique solutions

Ignores the “internal logic” of the encoding.

*Crucial for Knowledge Compilation: Can we compile a circuit that supports **Existential Quantification** efficiently?*

Outline

1 How to model using SAT

- SAT
- #SAT
- $\exists\#\text{SAT}$
 - Definition
 - Modeling
- Weighted #SAT

2 Knowledge Compilation

3 d-DNNF Queries

4 Conclusion and References

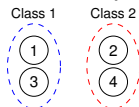
Revisiting Zykov: Partitions vs. Colorings

What are we actually counting?

Recall the Zykov variables s_{ij} (true if i, j share a color). A satisfying assignment of Σ_{Zykov} represents a **partition** of V into $m \leq k$ non-empty sets (equivalence classes).

- **The Discrepancy:** One partition into m color classes corresponds to multiple ways to assign k actual colors.

One Partition ($m = 2$)



Ways to color with $k = 3$:

Pick 2 colors out of 3, then arrange:

$$P(3, 2) = \frac{3!}{(3-2)!} = 6 \text{ colorings}$$

The Problem

If a partition uses m color classes, it represents $k!/(k-m)!$ colorings. A standard #SAT count on Σ_{Zykov} treats every partition as 1, regardless of m .

Solution: Assign a **weight** to each model based on the number of color classes m .

Outline

1 How to model using SAT

- SAT
- #SAT
- $\exists\#\text{SAT}$
- Weighted #SAT
 - Definition
 - Modeling

2 Knowledge Compilation

3 d-DNNF Queries

4 Conclusion and References

Outline

1 How to model using SAT

- SAT
- #SAT
- $\exists\#\text{SAT}$
- **Weighted #SAT**
 - Definition
 - Modeling

2 Knowledge Compilation

3 d-DNNF Queries

4 Conclusion and References

Weighted Model Counting

Weighted CNF Formula

- Let $V = \{v_1, \dots, v_n\}$ be a set of Boolean variables.
- Let $L = \{v_i, \bar{v}_i \mid v_i \in V\}$ be the set of literals over V .
- A **wCNF formula** over V is a pair (F, w) such that:
 - F is a conjunction of clauses,
 - w is a map from L to $\mathbb{Q}$ (i.e., a weighting of literals)

Weighted Model Counting

- The weight of a variable assignment $\mathbf{v} \in \{0, 1\}^n$ is the product of weights of literals which are true in $\mathbf{v}$:

$$W(\mathbf{v}) = \prod_{\ell \in L, \mathbf{v} \models \ell} w(\ell)$$

- The weighted model count of a wCNF formula is the sum of weights of the models of F :

$$W(F) = \sum_{\mathbf{v} \in \{0, 1\}^n, \mathbf{v} \models F} W(\mathbf{v})$$

Outline

1 How to model using SAT

- SAT
- #SAT
- $\exists\#\text{SAT}$
- **Weighted #SAT**
 - Definition
 - **Modeling**

2 Knowledge Compilation

3 d-DNNF Queries

4 Conclusion and References

Questions

Qualitative (Decision or Optimization)

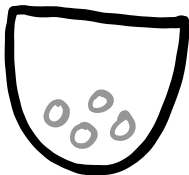
- Does the problem have a (optimal/preferred) solution?
- Output a reason for a decision.

Quantitative Questions

- How many solutions does the problem have?
- How likely is an occurrence?

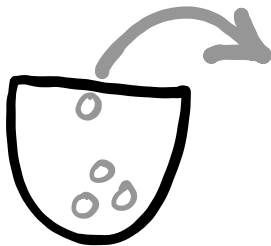
Bayesian Inference as Counting Problem

Example



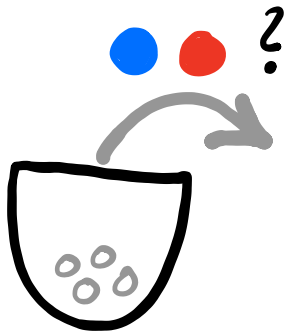
Bayesian Inference as Counting Problem

Example



Bayesian Inference as Counting Problem

Example



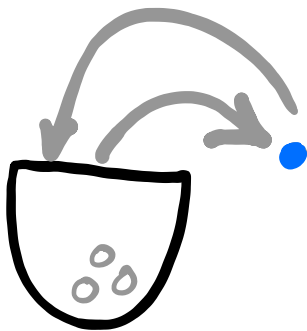
Bayesian Inference as Counting Problem

Example



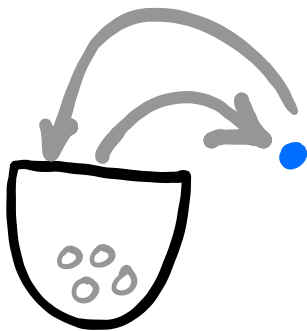
Bayesian Inference as Counting Problem

Example



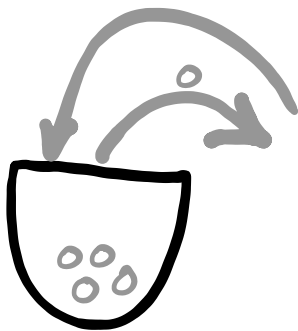
Bayesian Inference as Counting Problem

Example



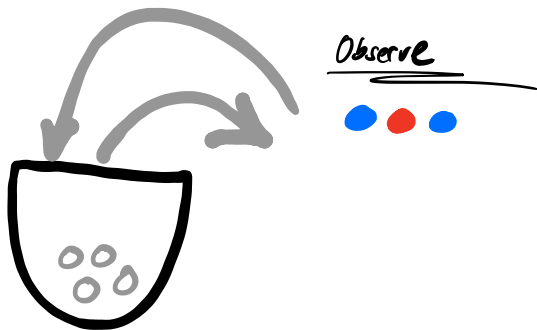
Bayesian Inference as Counting Problem

Example



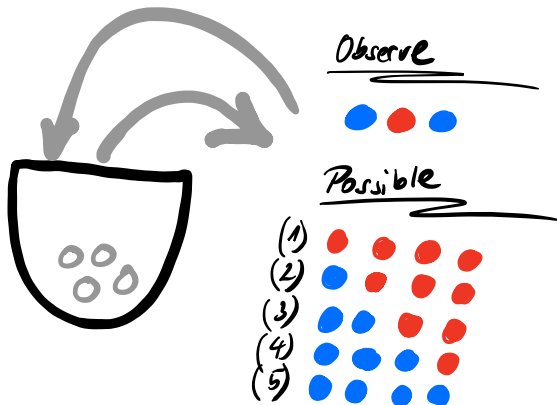
Bayesian Inference as Counting Problem

Example



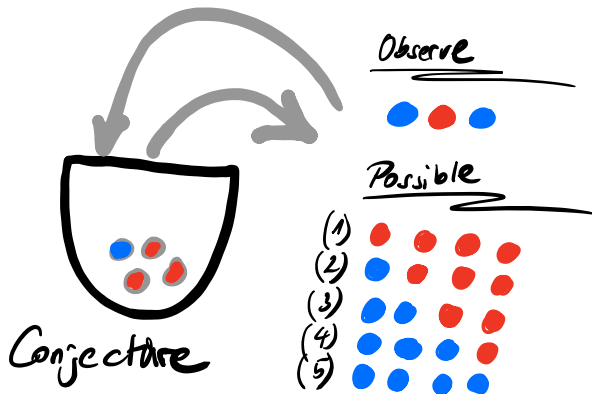
Bayesian Inference as Counting Problem

Example



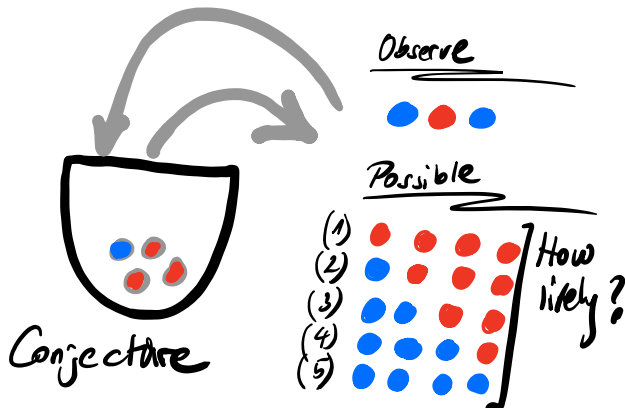
Bayesian Inference as Counting Problem

Example



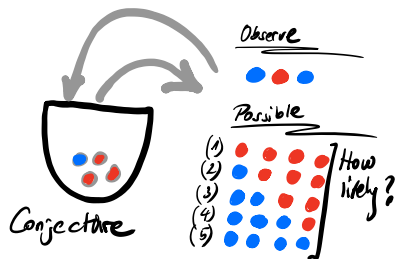
Bayesian Inference as Counting Problem

Example



Bayesian Inference as Counting Problem

Example

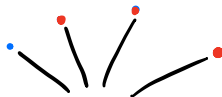


- How "likely"
- ⇒ Notion of Probability
- How does this relate to counting?

Bayesian Inference as Counting Problem

What possible combinations can we obtain?

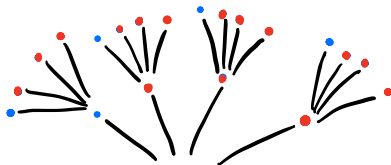
Conjecture



Bayesian Inference as Counting Problem

What possible combinations can we obtain?

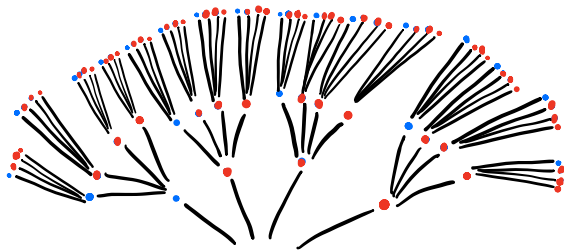
Conjecture



Bayesian Inference as Counting Problem

What possible combinations can we obtain?

Conjecture



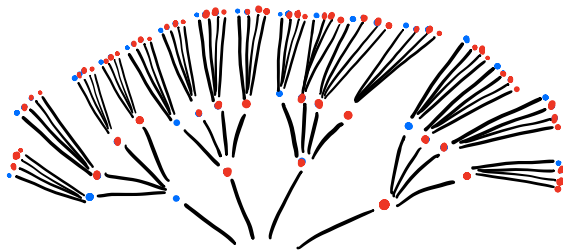
Bayesian Inference as Counting Problem

What possible combinations can we obtain?

Conjecture

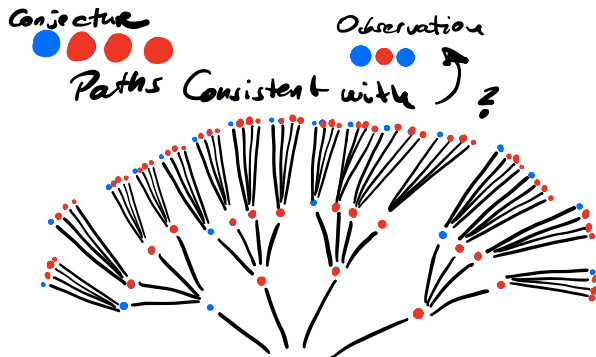


Observation



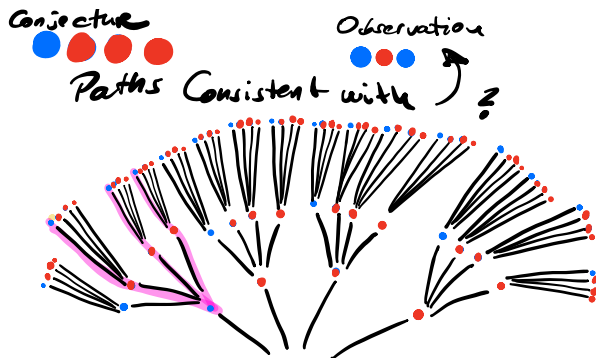
Bayesian Inference as Counting Problem

What possible combinations can we obtain?



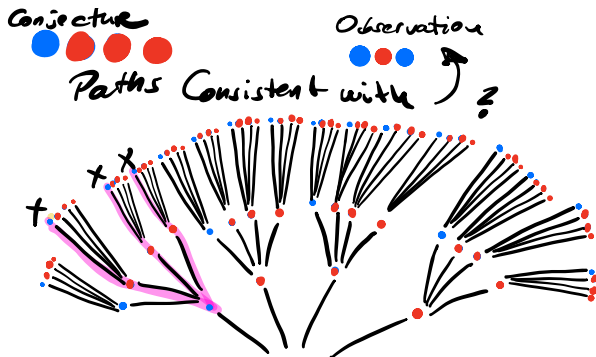
Bayesian Inference as Counting Problem

What possible combinations can we obtain?



Bayesian Inference as Counting Problem

What possible combinations can we obtain?



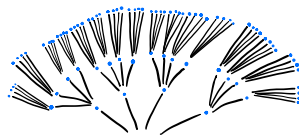
Bayesian Inference as Counting Problem

What possible combinations can we obtain?

Observation
● ● ●

Conjecture
● ● ● ●

0 paths



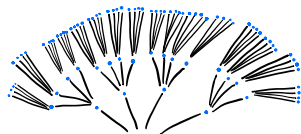
Bayesian Inference as Counting Problem

What possible combinations can we obtain?

Observation
● ● ●

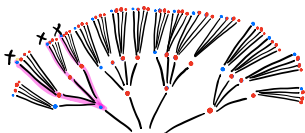
Conjecture
● ● ● ●

0 paths



Conjecture
● ● ● ●

3 paths



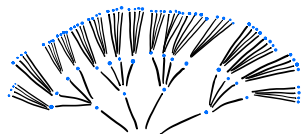
Bayesian Inference as Counting Problem

What possible combinations can we obtain?

Observation
● ● ●

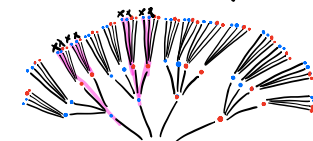
Conjecture
● ● ● ●

0 paths



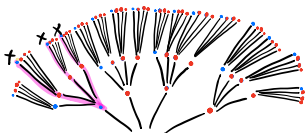
Conjecture
● ● ● ●

8 paths



Conjecture
● ● ● ●

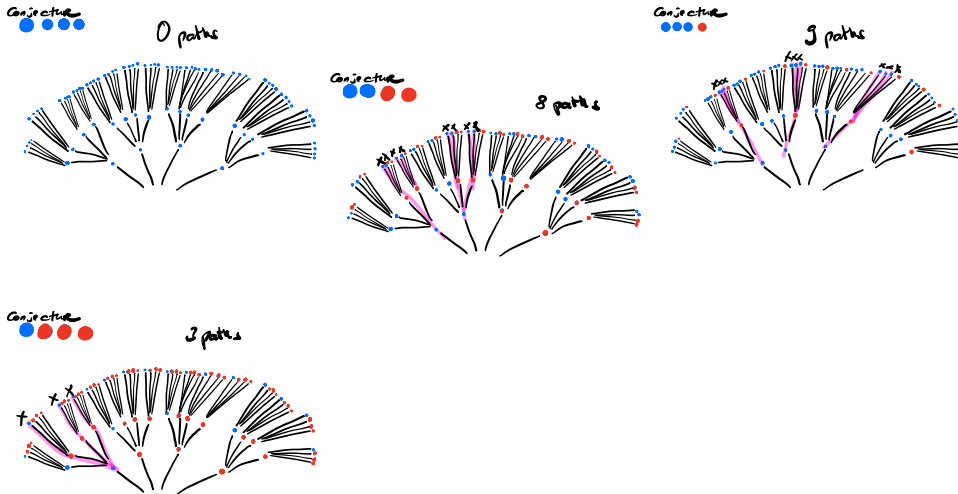
3 paths



Bayesian Inference as Counting Problem

What possible combinations can we obtain?

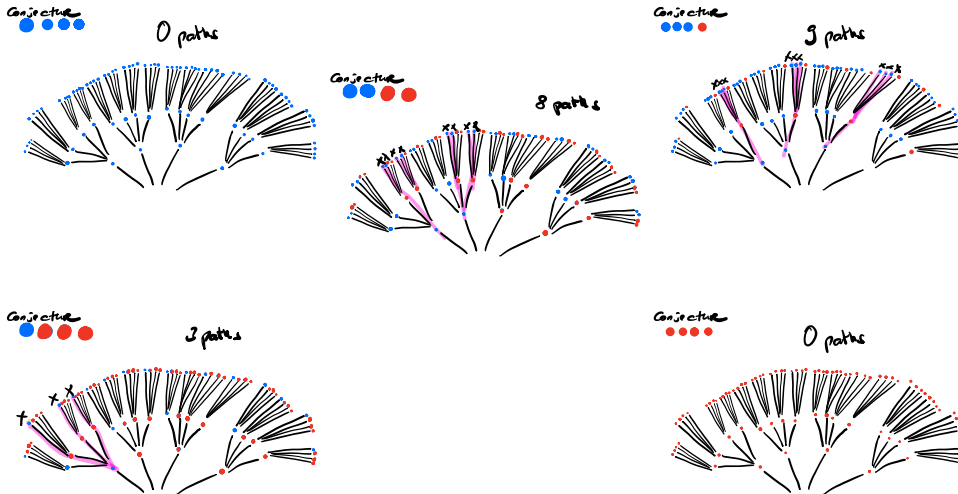
Observation
● ● ●



Bayesian Inference as Counting Problem

What possible combinations can we obtain?

Observation
● ● ●



Bayesian Inference as Counting Problem

What possible combinations can we obtain?

Conjecture	Possible Paths for $\langle \bullet \bullet \bullet \rangle$
$\begin{bmatrix} \bullet & \bullet & \bullet & \bullet \end{bmatrix}$	0
$\begin{bmatrix} \bullet & \bullet & \bullet & \bullet \end{bmatrix}$	9
$\begin{bmatrix} \bullet & \bullet & \bullet & \bullet \end{bmatrix}$	8
$\begin{bmatrix} \bullet & \bullet & \bullet & \bullet \end{bmatrix}$	3
$\begin{bmatrix} \bullet & \bullet & \bullet & \bullet \end{bmatrix}$	0
Total:	20

Bayesian Inference as Counting Problem

What possible combinations can we obtain?

Conjecture	Possible Paths for $\langle \bullet \bullet \bullet \rangle$
$[\bullet \bullet \bullet \bullet]$	0
$[\bullet \bullet \bullet \bullet]$	9
$[\bullet \bullet \bullet \bullet]$	8
$[\bullet \bullet \bullet \bullet]$	3
$[\bullet \bullet \bullet \bullet]$	0
Total:	20

Probability: relationship between actual and possible occurrence

Bayesian Inference as Counting Problem

What possible combinations can we obtain?

Conjecture	Possible Paths for $\langle \bullet \bullet \bullet \rangle$
$[\bullet \bullet \bullet \bullet]$	0
$[\bullet \bullet \bullet \bullet]$	9
$[\bullet \bullet \bullet \bullet]$	8
$[\bullet \bullet \bullet \bullet]$	3
$[\bullet \bullet \bullet \bullet]$	0
Total:	20

$$\text{Probability } [\bullet \bullet \bullet \bullet] = \frac{3}{0+9+8+3+0} = \frac{3}{20}$$

Bayesian Inference as Counting Problem

What possible combinations can we obtain?

Conjecture	Possible Paths for $\langle \bullet \bullet \bullet \rangle$	Probability
$[\bullet \bullet \bullet \bullet]$	0	$0/20 = 0$
$[\bullet \bullet \bullet \bullet]$	9	$9/20 = 0.45$
$[\bullet \bullet \bullet \bullet]$	8	$8/20 = 0.40$
$[\bullet \bullet \bullet \bullet]$	3	$3/20 = 0.15$
$[\bullet \bullet \bullet \bullet]$	0	$0/20 = 0$
Total:	20	1

Probabilistic Inference



Probabilistic Inference



■ Observations:

- It is cloudy
- The grass is wet
- ...

Probabilistic Inference



■ Observations:

- It is cloudy
- The grass is wet
- ...

day	cloudy	wet grass
1	1	1
2	1	1
3	1	0
4	0	0

$$\mathbb{P}(\text{wet grass}|\text{cloudy}) = \frac{2}{3}$$

Probabilistic Inference



■ Observations:

- It is cloudy
- The grass is wet
- ...

day	cloudy	wet grass
1	1	1
2	1	1
3	1	0
4	0	0

$$\mathbb{P}(\text{wet grass}|\text{cloudy}) = \frac{2}{3}$$

- Modeling a problem using a **joint distribution** on a set of random variables

Probabilistic Inference

- Let $\mathbf{X}$ be a set of random variables over finite domains, here mostly $\{0, 1\}$
- $\text{Inst}(\mathbf{X})$ is the set of all complete assignments to $\mathbf{X}$
- An **instantiation** $\mathbf{x} \in \text{Inst}(\mathbf{X})$ assigns to each variable $X_i \in \mathbf{X}$ a value (from its domain)
- A **probability mass function** $\mathbb{P}$ describes probabilities of instantiations, i.e., a function $\mathbb{P} : \text{Inst}(\mathbf{X}) \rightarrow [0, 1]$ and is a **joint distribution** over $\mathbf{X}$ if $\sum_{\mathbf{x} \in \text{Inst}(\mathbf{X})} \mathbb{P}(\mathbf{x}) = 1$
- **Evidence** is a partial instantiation, an assignment to only some variables of $\mathbf{X}$

Cloudy	Rain	Sprinkler	Wet	$\mathbb{P}(C, R, S, W)$
0	0	0	0	0.225
0	0	0	1	0
0	0	1	0	0.045
0	0	1	1	0.18
0	1	0	0	0.0025
0	1	0	1	0.0225
0	1	1	0	0
0	1	1	1	0.025
1	0	0	0	0.09
1	0	0	1	0
1	0	1	0	0.002
1	0	1	1	0.008
1	1	0	0	0.036
1	1	0	1	0.324
1	1	1	0	0
1	1	1	1	0.04

Probabilistic Inference

- Let $\mathbf{X}$ be a set of random variables over finite domains, here mostly $\{0, 1\}$
- $\text{Inst}(\mathbf{X})$ is the set of all complete assignments to $\mathbf{X}$
- An **instantiation** $\mathbf{x} \in \text{Inst}(\mathbf{X})$ assigns to each variable $X_i \in \mathbf{X}$ a value (from its domain)
- A **probability mass function** $\mathbb{P}$ describes probabilities of instantiations, i.e., a function $\mathbb{P} : \text{Inst}(\mathbf{X}) \rightarrow [0, 1]$ and is a **joint distribution** over $\mathbf{X}$ if $\sum_{\mathbf{x} \in \text{Inst}(\mathbf{X})} \mathbb{P}(\mathbf{x}) = 1$
- **Evidence** is a partial instantiation, an assignment to only some variables of $\mathbf{X}$

Cloudy	Rain	Sprinkler	Wet	$\mathbb{P}(C, R, S, W)$
0	0	0	0	0.225
0	0	0	1	0
0	0	1	0	0.045
0	0	1	1	0.18
0	1	0	0	0.0025
0	1	0	1	0.0225
0	1	1	0	0
0	1	1	1	0.025
1	0	0	0	0.09
1	0	0	1	0
1	0	1	0	0.002
1	0	1	1	0.008
1	1	0	0	0.036
1	1	0	1	0.324
1	1	1	0	0
1	1	1	1	0.04

- **probability of instantiation $\mathbf{x}$**

$$\mathbb{P}(C = 0, R = 1, S = 0, W = 1) = 0.0225$$

$$\mathbb{P}(C_0, R_1, S_0, W_1) = 0.0225$$

Probabilistic Inference

- Let $\mathbf{X}$ be a set of random variables over finite domains, here mostly $\{0, 1\}$
- $\text{Inst}(\mathbf{X})$ is the set of all complete assignments to $\mathbf{X}$
- An **instantiation** $\mathbf{x} \in \text{Inst}(\mathbf{X})$ assigns to each variable $X_i \in \mathbf{X}$ a value (from its domain)
- A **probability mass function** $\mathbb{P}$ describes probabilities of instantiations, i.e., a function $\mathbb{P} : \text{Inst}(\mathbf{X}) \rightarrow [0, 1]$ and is a **joint distribution** over $\mathbf{X}$ if $\sum_{\mathbf{x} \in \text{Inst}(\mathbf{X})} \mathbb{P}(\mathbf{x}) = 1$
- **Evidence** is a partial instantiation, an assignment to only some variables of $\mathbf{X}$

Cloudy	Rain	Sprinkler	Wet	$\mathbb{P}(C, R, S, W)$
0	0	0	0	0.225
0	0	0	1	0
0	0	1	0	0.045
0	0	1	1	0.18
0	1	0	0	0.0025
0	1	0	1	0.0225
0	1	1	0	0
0	1	1	1	0.025
1	0	0	0	0.09
1	0	0	1	0
1	0	1	0	0.002
1	0	1	1	0.008
1	1	0	0	0.036
1	1	0	1	0.324
1	1	1	0	0
1	1	1	1	0.04

■ probability of instantiation $\mathbf{x}$

$$\mathbb{P}(C = 0, R = 1, S = 0, W = 1) = 0.0225$$

$$\mathbb{P}(C_0, R_1, S_0, W_1) = 0.0225$$

■ probability of evidence e

$$\begin{aligned}\mathbb{P}(S_0, W_1) &= 0 + 0.0225 + 0 + 0.324 \\ &= 0.3465\end{aligned}$$

Probabilistic Inference

- Let $\mathbf{X}$ be a set of random variables over finite domains, here mostly $\{0, 1\}$
- $\text{Inst}(\mathbf{X})$ is the set of all complete assignments to $\mathbf{X}$
- An **instantiation** $\mathbf{x} \in \text{Inst}(\mathbf{X})$ assigns to each variable $X_i \in \mathbf{X}$ a value (from its domain)
- A **probability mass function** $\mathbb{P}$ describes probabilities of instantiations, i.e., a function $\mathbb{P} : \text{Inst}(\mathbf{X}) \rightarrow [0, 1]$ and is a **joint distribution** over $\mathbf{X}$ if $\sum_{\mathbf{x} \in \text{Inst}(\mathbf{X})} \mathbb{P}(\mathbf{x}) = 1$
- **Evidence** is a partial instantiation, an assignment to only some variables of $\mathbf{X}$

Cloudy	Rain	Sprinkler	Wet	$\mathbb{P}(C, R, S, W)$
0	0	0	0	0.225
0	0	0	1	0
0	0	1	0	0.045
0	0	1	1	0.18
0	1	0	0	0.0025
0	1	0	1	0.0225
0	1	1	0	0
0	1	1	1	0.025
1	0	0	0	0.09
1	0	0	1	0
1	0	1	0	0.002
1	0	1	1	0.008
1	1	0	0	0.036
1	1	0	1	0.324
1	1	1	0	0
1	1	1	1	0.04

■ probability of instantiation $\mathbf{x}$

$$\mathbb{P}(C = 0, R = 1, S = 0, W = 1) = 0.0225$$

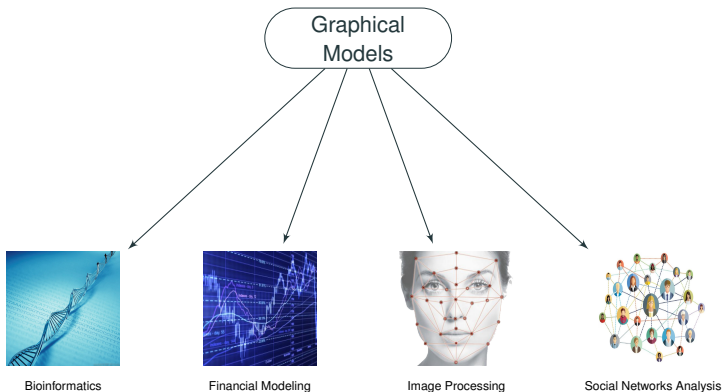
$$\mathbb{P}(C_0, R_1, S_0, W_1) = 0.0225$$

■ probability of evidence e

$$\begin{aligned}\mathbb{P}(S_0, W_1) &= 0 + 0.0225 + 0 + 0.324 \\ &= 0.3465\end{aligned}$$

- The size of the joint distribution is exponential in the number of variables!

Graphical Models



Graphical models are a well-studied framework for representing high-dimensional probability distributions, with a wide spectrum of applications.

Graphical Models

Conditional probability table

R	C	$\mathbb{P}(R C)$
0	0	0.9
1	0	0.1
0	1	0.2
1	1	0.8

Prior probability distribution

C	$\mathbb{P}(C)$
0	0.5
1	0.5

S	C	$\mathbb{P}(S C)$
0	0	0.5
1	0	0.5
0	1	0.9
1	1	0.1

A Bayesian Network

Joint distribution

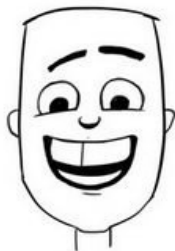
$$P(C, R, S, W) = P(C) \cdot P(R|C) \cdot P(S|C) \cdot P(W|S, R)$$

W	S	R	$\mathbb{P}(W S, R)$
0	0	0	1.0
1	0	0	0.0
0	0	1	0.1
1	0	1	0.9
0	1	0	0.2
1	1	0	0.8
0	1	1	0.0
1	1	1	1.0

Graphical models are commonly separated into:

- **directed** models (a.k.a. Bayesian networks), with conditional probability tables (CPTs) over a directed (acyclic) graph,
- **undirected** models (a.k.a. Markov networks), with energy functions over the cliques of an undirected graph.

Bayesian network



Hey! What's the probability that it's not cloudy, it rains, the grass is wet, and the sprinkler is off?

The semantics of Bayesian networks implies the following joint distribution:

$$\mathbb{P}(x_1, x_2, \dots, x_n) = \prod_i \mathbb{P}(x_i | u_i)$$

Bayesian network



Hey! What's the probability that it's not cloudy, it rains, the grass is wet, and the sprinkler is off?

$$\begin{aligned}\mathbb{P}(c_0, r_1, s_0, w_1) &= \mathbb{P}(c_0) \cdot \mathbb{P}(r_1|c_0) \cdot \mathbb{P}(s_0|c_0) \cdot \mathbb{P}(w_1|s_0, r_1) \\ &= 0.5 \cdot 0.1 \cdot 0.5 \cdot 0.9 \\ &= 0.0225\end{aligned}$$

The semantics of Bayesian networks implies the following joint distribution:

$$\mathbb{P}(x_1, x_2, \dots, x_n) = \prod_i \mathbb{P}(x_i | u_i)$$

Probabilistic Inference



What are the chances of having a wet grass when it's cloudy?

Product rule: $\mathbb{P}(A, X) = \mathbb{P}(A|X) \cdot \mathbb{P}(X)$

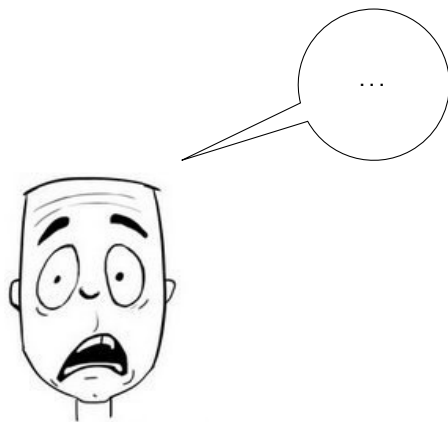
$$\mathbb{P}(w_1 | c_1) = \frac{\mathbb{P}(w_1, c_1)}{\mathbb{P}(c_1)}$$

Bayes' rule: $\mathbb{P}(A|X) = \frac{\mathbb{P}(X|A) \cdot \mathbb{P}(A)}{\mathbb{P}(X)}$

$$\mathbb{P}(c_1 | s_1) = \frac{\mathbb{P}(s_1, c_1) \cdot \mathbb{P}(c_1)}{\mathbb{P}(s_1)}$$

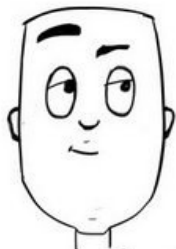
Probabilistic inference is a basic task for handling more complex queries:

- Conditional probabilities: evaluating the probability of an event $\mathbf{y}$ given an evidence $\mathbf{x}$;
- Most probable explanations: given an evidence $\mathbf{x}$, find an assignment $\mathbf{y}$ over the complementary variables that maximizes $\mathbb{P}(\mathbf{y} | \mathbf{x})$;
- ...



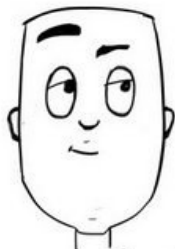
Unfortunately, probabilistic inference is **#P-complete** in graphical models, including both Bayesian networks and Markov networks.

Probabilistic Inference to Weighted Model Counting



So ... Can we **reduce** the problem of probabilistic inference to Weighted Model Counting?

Probabilistic Inference to Weighted Model Counting



So ... Can we **reduce** the problem of probabilistic inference to Weighted Model Counting?

Yes! The idea is to find a **translation function** τ mapping

- any graphical model (Bayes or Markov net) N to a weighted CNF formula $\tau(N)$, and
- any partial assignment $\mathbf{x}$ to a term $\tau(\mathbf{x})$,

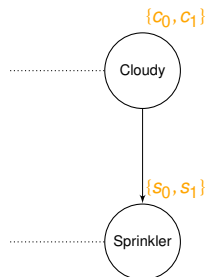
such that

$$\mathbb{P}_N(\mathbf{x}) = \frac{W[\tau(N) \wedge \tau(\mathbf{x})]}{W[\tau(N)]}$$

where $W[\tau(N)]$ is the **partition constant** ($= 1$ for BNs).

C	$\mathbb{P}(C)$
0	0.5
1	0.5

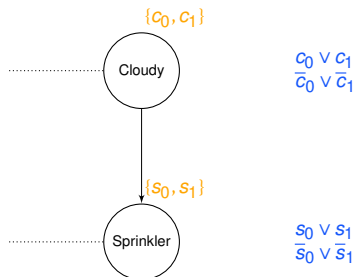
S	C	$\mathbb{P}(S C)$
0	0	0.5
1	0	0.5
0	1	0.9
1	1	0.1



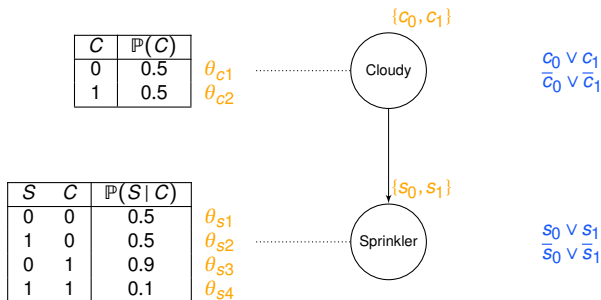
- An indicator variable v_{ij} for each random variable X_i and domain value $j \in D(X_i)$;

C	$\mathbb{P}(C)$
0	0.5
1	0.5

S	C	$\mathbb{P}(S C)$
0	0	0.5
1	0	0.5
0	1	0.9
1	1	0.1



- An indicator variable v_{ij} for each random variable X_i and domain value $j \in D(X_i)$;
- A set of indicator clauses for each random variable X_i (direct encoding);



- An indicator variable v_{ij} for each random variable X_i and domain value $j \in D(X_i)$;
- A set of indicator clauses for each random variable X_i (direct encoding);
- A parameter variable θ_{ir} for each row r in the table of X_i , and a weight $w(\theta_{ir}) = \mathbb{P}(x_i | \mathbf{x}_r)$;

$$c_0 \leftrightarrow \theta_{c1}$$

$$c_1 \leftrightarrow \theta_{c2}$$

C	$\mathbb{P}(C)$
0	0.5
1	0.5

 θ_{c1}
 θ_{c2}

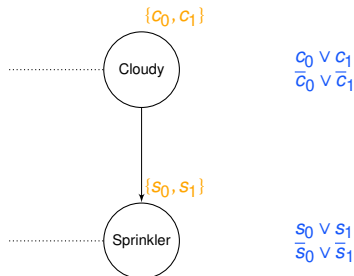
$$s_0 \wedge c_0 \leftrightarrow \theta_{s1}$$

$$s_1 \wedge c_0 \leftrightarrow \theta_{s2}$$

$$s_0 \wedge c_1 \leftrightarrow \theta_{s3}$$

$$s_1 \wedge c_1 \leftrightarrow \theta_{s4}$$

S	C	$\mathbb{P}(S C)$
0	0	0.5
1	0	0.5
0	1	0.9
1	1	0.1

 θ_{s1}
 θ_{s2}
 θ_{s3}
 θ_{s4}


- An indicator variable v_{ij} for each random variable X_i and domain value $j \in D(X_i)$;
- A set of indicator clauses for each random variable X_i (direct encoding);
- A parameter variable θ_{ir} for each row r in the table of X_i , and a weight $w(\theta_{ir}) = \mathbb{P}(x_i | \mathbf{x}_r)$;
- A set of equivalences describing the semantics of each row in the table of X_i .

Local Structure

- In practice, variables are not necessary binary and CPTs can be quite large

A	B	C	$\mathbb{P}(c a,b)$
a_1	b_1	c_1	0
a_1	b_1	c_2	0.5
a_1	b_1	c_3	0.5
a_1	b_2	c_1	0.2
a_1	b_2	c_2	0.3
a_1	b_2	c_3	0.5
a_2	b_1	c_1	0
a_2	b_1	c_2	0
a_2	b_1	c_3	1
a_2	b_2	c_1	0.2
a_2	b_2	c_2	0.3
a_2	b_2	c_3	0.5

- Thus, the generated CNF encoding can be large, challenging state-of-the-art weighted model counters

Remove Inconsistent Assignments

A	B	C	$\mathbb{P}(c a,b)$
a_1	b_1	c_1	0
a_1	b_1	c_2	0.5
a_1	b_1	c_3	0.5
a_1	b_2	c_1	0.2
a_1	b_2	c_2	0.3
a_1	b_2	c_3	0.5
a_2	b_1	c_1	0
a_2	b_1	c_2	0
a_2	b_1	c_3	1
a_2	b_2	c_1	0.2
a_2	b_2	c_2	0.3
a_2	b_2	c_3	0.5

Remove Inconsistent Assignments

A	B	C	$\mathbb{P}(c a,b)$
a_1	b_1	c_1	0
a_1	b_1	c_2	0.5
a_1	b_1	c_3	0.5
a_1	b_2	c_1	0.2
a_1	b_2	c_2	0.3
a_1	b_2	c_3	0.5
a_2	b_1	c_1	0
a_2	b_1	c_2	0
a_2	b_1	c_3	1
a_2	b_2	c_1	0.2
a_2	b_2	c_2	0.3
a_2	b_2	c_3	0.5

- The weight of a model is obtained by multiplying the weights of its positive parameters
- Then, all models which contain a positive parameter with a weight of 0 can be removed

Remove Inconsistent Assignments

A	B	C	$\mathbb{P}(c a,b)$
a_1	b_1	c_1	0
a_1	b_1	c_2	0.5
a_1	b_1	c_3	0.5
a_1	b_2	c_1	0.2
a_1	b_2	c_2	0.3
a_1	b_2	c_3	0.5
a_2	b_1	c_1	0
a_2	b_1	c_2	0
a_2	b_1	c_3	1
a_2	b_2	c_1	0.2
a_2	b_2	c_2	0.3
a_2	b_2	c_3	0.5

- The weight of a model is obtained by multiplying the weights of its positive parameters
- Then, all models which contain a positive parameter with a weight of 0 can be removed

$$\neg a_1 \vee \neg b_1 \vee \neg c_1 \quad \neg a_2 \vee \neg b_1 \vee \neg c_1 \quad \neg a_2 \vee \neg b_1 \vee \neg c_2$$

Equal Parameters

A	B	C	$\mathbb{P}(c a,b)$
a_1	b_1	c_1	0
a_1	b_1	c_2	0.5
a_1	b_1	c_3	0.5
a_1	b_2	c_1	0.2
a_1	b_2	c_2	0.3
a_1	b_2	c_3	0.5
a_2	b_1	c_1	0
a_2	b_1	c_2	0
a_2	b_1	c_3	1
a_2	b_2	c_1	0.2
a_2	b_2	c_2	0.3
a_2	b_2	c_3	0.5

Equal Parameters

A	B	C	$\mathbb{P}(c a,b)$
a_1	b_1	c_1	0
a_1	b_1	c_2	0.5
a_1	b_1	c_3	0.5
a_1	b_2	c_1	0.2
a_1	b_2	c_2	0.3
a_1	b_2	c_3	0.5
a_2	b_1	c_1	0
a_2	b_1	c_2	0
a_2	b_1	c_3	1
a_2	b_2	c_1	0.2
a_2	b_2	c_2	0.3
a_2	b_2	c_3	0.5

- ENC1 will generate 9 parameter variables
- Group parameters that are equal and use a unique parameter variable by group

$$a_1 \wedge b_1 \wedge c_2 \rightarrow \theta_{c1}$$

$$a_1 \wedge b_1 \wedge c_3 \rightarrow \theta_{c1}$$

$$a_1 \wedge b_2 \wedge c_3 \rightarrow \theta_{c1}$$

$$a_2 \wedge b_2 \wedge c_3 \rightarrow \theta_{c1}$$

with $w(\theta_{c1}) = 0.5$ and we have to filter the models

Decomposability

- Model counters (and compilers) typically [look for dependent components](#)
- The translation into CNF may obfuscate the recognition of independent components

A	B	C	$\mathbb{P}(c a,b)$
a_1	b_1	c_1	0.25
a_1	b_1	c_2	0.25
a_1	b_2	c_1	0.25
a_1	b_2	c_2	0.25
a_2	b_1	c_1	0
a_2	b_1	c_2	0.5
a_2	b_2	c_1	0.2
a_2	b_2	c_2	0.3

$$\begin{array}{ll}
 a_1 \wedge b_1 \wedge c_1 \rightarrow \theta_{c1} & \neg a_2 \vee \neg b_1 \vee \neg c_1 \\
 a_1 \wedge b_1 \wedge c_2 \rightarrow \theta_{c1} & a_2 \wedge b_1 \wedge c_2 \rightarrow \theta_{c2} \\
 a_1 \wedge b_2 \wedge c_1 \rightarrow \theta_{c1} & a_2 \wedge b_2 \wedge c_1 \rightarrow \theta_{c3} \\
 a_1 \wedge b_2 \wedge c_2 \rightarrow \theta_{c1} & a_2 \wedge b_2 \wedge c_2 \rightarrow \theta_{c4}
 \end{array}$$

B	E	D	$\mathbb{P}(d b,e)$
b_1	e_1	d_1	0.2
b_1	e_1	d_2	0.3
b_1	e_2	d_1	0.1
b_1	e_2	d_2	0.4
b_2	e_1	d_1	0.1
b_2	e_1	d_2	0.4
b_2	e_2	d_1	0.2
b_2	e_2	d_2	0.3

$$\begin{array}{ll}
 b_1 \wedge e_1 \wedge d_1 \rightarrow \theta_{d5} & b_2 \wedge e_1 \wedge d_1 \rightarrow \theta_{d7} \\
 b_1 \wedge e_1 \wedge d_2 \rightarrow \theta_{d6} & b_2 \wedge e_1 \wedge d_2 \rightarrow \theta_{d8} \\
 b_1 \wedge e_2 \wedge d_1 \rightarrow \theta_{d7} & b_2 \wedge e_2 \wedge d_1 \rightarrow \theta_{d5} \\
 b_1 \wedge e_2 \wedge d_2 \rightarrow \theta_{d8} & b_2 \wedge e_2 \wedge d_2 \rightarrow \theta_{d6}
 \end{array}$$

Decomposability

- Model counters (and compilers) typically [look for dependent components](#)
- The translation into CNF may obfuscate the recognition of independent components

A	B	C	$\mathbb{P}(c a,b)$
a_1	b_1	c_1	0.25
a_1	b_1	c_2	0.25
a_1	b_2	c_1	0.25
a_1	b_2	c_2	0.25
a_2	b_1	c_1	0
a_2	b_1	c_2	0.5
a_2	b_2	c_1	0.2
a_2	b_2	c_2	0.3

B	E	D	$\mathbb{P}(d b,e)$
b_1	e_1	d_1	0.2
b_1	e_1	d_2	0.3
b_1	e_2	d_1	0.1
b_1	e_2	d_2	0.4
b_2	e_1	d_1	0.1
b_2	e_1	d_2	0.4
b_2	e_2	d_1	0.2
b_2	e_2	d_2	0.3

$$\begin{array}{ll}
 a_1 \wedge b_1 \wedge c_1 \rightarrow \theta_{c1} & \neg a_2 \vee \neg b_1 \vee \neg c_1 \\
 a_1 \wedge b_1 \wedge c_2 \rightarrow \theta_{c1} & a_2 \wedge b_1 \wedge c_2 \rightarrow \theta_{c2} \\
 a_1 \wedge b_2 \wedge c_1 \rightarrow \theta_{c1} & a_2 \wedge b_2 \wedge c_1 \rightarrow \theta_{c3} \\
 a_1 \wedge b_2 \wedge c_2 \rightarrow \theta_{c1} & a_2 \wedge b_2 \wedge c_2 \rightarrow \theta_{c4}
 \end{array}$$

$$\begin{array}{ll}
 b_1 \wedge e_1 \wedge d_1 \rightarrow \theta_{d5} & b_2 \wedge e_1 \wedge d_1 \rightarrow \theta_{d7} \\
 b_1 \wedge e_1 \wedge d_2 \rightarrow \theta_{d6} & b_2 \wedge e_1 \wedge d_2 \rightarrow \theta_{d8} \\
 b_1 \wedge e_2 \wedge d_1 \rightarrow \theta_{d7} & b_2 \wedge e_2 \wedge d_1 \rightarrow \theta_{d5} \\
 b_1 \wedge e_2 \wedge d_2 \rightarrow \theta_{d8} & b_2 \wedge e_2 \wedge d_2 \rightarrow \theta_{d6}
 \end{array}$$

- [Computing implicants](#) before translating into CNF [promotes the identification of independent components](#)

Decomposability

- Model counters (and compilers) typically [look for dependent components](#)
- The translation into CNF may obfuscate the recognition of independent components

A	B	C	$\mathbb{P}(c a,b)$
a_1	b_1	c_1	0.25
a_1	b_1	c_2	0.25
a_1	b_2	c_1	0.25
a_1	b_2	c_2	0.25
a_2	b_1	c_1	0
a_2	b_1	c_2	0.5
a_2	b_2	c_1	0.2
a_2	b_2	c_2	0.3

B	E	D	$\mathbb{P}(d b,e)$
b_1	e_1	d_1	0.2
b_1	e_1	d_2	0.3
b_1	e_2	d_1	0.1
b_1	e_2	d_2	0.4
b_2	e_1	d_1	0.1
b_2	e_1	d_2	0.4
b_2	e_2	d_1	0.2
b_2	e_2	d_2	0.3

$$\begin{array}{ll}
 a_1 \wedge b_1 \wedge c_1 \rightarrow \theta_{c1} & \neg a_2 \vee \neg b_1 \vee \neg c_1 \\
 a_1 \wedge b_1 \wedge c_2 \rightarrow \theta_{c1} & a_2 \wedge b_1 \wedge c_2 \rightarrow \theta_{c2} \\
 a_1 \wedge b_2 \wedge c_1 \rightarrow \theta_{c1} & a_2 \wedge b_2 \wedge c_1 \rightarrow \theta_{c3} \\
 a_1 \wedge b_2 \wedge c_2 \rightarrow \theta_{c1} & a_2 \wedge b_2 \wedge c_2 \rightarrow \theta_{c4}
 \end{array}$$

$$\begin{array}{ll}
 b_1 \wedge e_1 \wedge d_1 \rightarrow \theta_{d5} & b_2 \wedge e_1 \wedge d_1 \rightarrow \theta_{d7} \\
 b_1 \wedge e_1 \wedge d_2 \rightarrow \theta_{d6} & b_2 \wedge e_1 \wedge d_2 \rightarrow \theta_{d8} \\
 b_1 \wedge e_2 \wedge d_1 \rightarrow \theta_{d7} & b_2 \wedge e_2 \wedge d_1 \rightarrow \theta_{d5} \\
 b_1 \wedge e_2 \wedge d_2 \rightarrow \theta_{d8} & b_2 \wedge e_2 \wedge d_2 \rightarrow \theta_{d6}
 \end{array}$$

- [Computing implicants](#) before translating into CNF [promotes the identification of independent components](#)

Decomposability

- Model counters (and compilers) typically [look for dependent components](#)
- The translation into CNF may obfuscate the recognition of independent components

A	B	C	$\mathbb{P}(c a,b)$
a_1	b_1	c_1	0.25
a_1	b_1	c_2	0.25
a_1	b_2	c_1	0.25
a_1	b_2	c_2	0.25
a_2	b_1	c_1	0
a_2	b_1	c_2	0.5
a_2	b_2	c_1	0.2
a_2	b_2	c_2	0.3

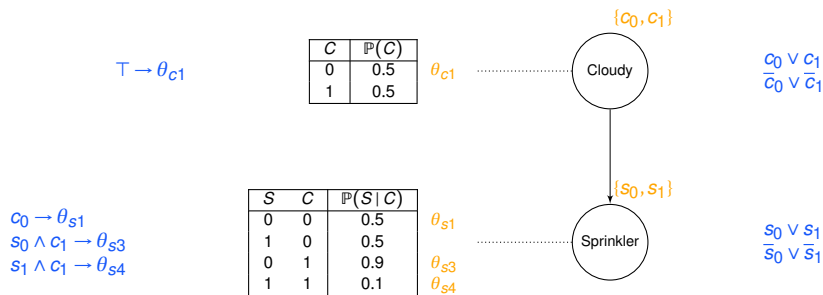
B	E	D	$\mathbb{P}(d b,e)$
b_1	e_1	d_1	0.2
b_1	e_1	d_2	0.3
b_1	e_2	d_1	0.1
b_1	e_2	d_2	0.4
b_2	e_1	d_1	0.1
b_2	e_1	d_2	0.4
b_2	e_2	d_1	0.2
b_2	e_2	d_2	0.3

$$\begin{aligned}
 a_1 &\rightarrow \theta_{c1} & \neg a_2 \vee \neg b_1 \vee \neg c_1 \\
 & & a_2 \wedge b_1 \wedge c_2 \rightarrow \theta_{c2} \\
 & & a_2 \wedge b_2 \wedge c_1 \rightarrow \theta_{c3} \\
 & & a_2 \wedge b_2 \wedge c_2 \rightarrow \theta_{c4}
 \end{aligned}$$

$$\begin{aligned}
 b_1 \wedge e_1 \wedge d_1 &\rightarrow \theta_{d5} & b_2 \wedge e_1 \wedge d_1 &\rightarrow \theta_{d7} \\
 b_1 \wedge e_1 \wedge d_2 &\rightarrow \theta_{d6} & b_2 \wedge e_1 \wedge d_2 &\rightarrow \theta_{d8} \\
 b_1 \wedge e_2 \wedge d_1 &\rightarrow \theta_{d7} & b_2 \wedge e_2 \wedge d_1 &\rightarrow \theta_{d5} \\
 b_1 \wedge e_2 \wedge d_2 &\rightarrow \theta_{d8} & b_2 \wedge e_2 \wedge d_2 &\rightarrow \theta_{d6}
 \end{aligned}$$

- [Computing implicants](#) before translating into CNF [promotes the identification of independent components](#)

Chavira and Darwiche's Translation (Chavira and Darwiche, 2005)



A more compact encoding of tables:

- Group rows with the same probability, and compress them (into a prime DNF);
- Use implications instead of equivalences.

Log Encoding (Bart et al., 2016)

- In the log encoding domains are represented with $m = \lceil \log_2(d) \rceil$ propositional variables
- Each of the 2^m combinations represents a possible assignment
- Introduce less indicator variables than the direct encoding

Log Encoding (Bart et al., 2016)

- In the log encoding domains are represented with $m = \lceil \log_2(d) \rceil$ propositional variables
- Each of the 2^m combinations represents a possible assignment
- Introduce less indicator variables than the direct encoding

green
blue
gray
purple
black
white

Log Encoding (Bart et al., 2016)

- In the log encoding domains are represented with $m = \lceil \log_2(d) \rceil$ propositional variables
- Each of the 2^m combinations represents a possible assignment
- Introduce less indicator variables than the direct encoding

green	0
blue	1
gray	2
purple	3
black	4
white	5

Log Encoding (Bart et al., 2016)

- In the log encoding domains are represented with $m = \lceil \log_2(d) \rceil$ propositional variables
- Each of the 2^m combinations represents a possible assignment
- Introduce less indicator variables than the direct encoding

3 propositional variables $\{c_2, c_1, c_0\}$

green	0
blue	1
gray	2
purple	3
black	4
white	5

Log Encoding (Bart et al., 2016)

- In the log encoding domains are represented with $m = \lceil \log_2(d) \rceil$ propositional variables
- Each of the 2^m combinations represents a possible assignment
- Introduce less indicator variables than the direct encoding

3 propositional variables $\{c_2, c_1, c_0\}$

green	0	$\neg c_2 \wedge \neg c_1 \wedge \neg c_0$
blue	1	$\neg c_2 \wedge \neg c_1 \wedge c_0$
gray	2	$\neg c_2 \wedge c_1 \wedge \neg c_0$
purple	3	$\neg c_2 \wedge c_1 \wedge c_0$
black	4	$c_2 \wedge \neg c_1 \wedge \neg c_0$
white	5	$c_2 \wedge \neg c_1 \wedge c_0$

Log Encoding (Bart et al., 2016)

- In the log encoding domains are represented with $m = \lceil \log_2(d) \rceil$ propositional variables
- Each of the 2^m combinations represents a possible assignment
- Introduce less indicator variables than the direct encoding

3 propositional variables $\{c_2, c_1, c_0\}$

green	0	$\neg c_2 \wedge \neg c_1 \wedge \neg c_0$
blue	1	$\neg c_2 \wedge \neg c_1 \wedge c_0$
gray	2	$\neg c_2 \wedge c_1 \wedge \neg c_0$
purple	3	$\neg c_2 \wedge c_1 \wedge c_0$
black	4	$c_2 \wedge \neg c_1 \wedge \neg c_0$
white	5	$c_2 \wedge \neg c_1 \wedge c_0$

- We need to exclude the values in excess with a logarithmic number of clauses

$$\neg c_2 \vee \neg c_1 \vee \neg c_0$$

$$\neg c_2 \vee \neg c_1 \vee c_0$$

- Domains having 2^d values don't need indicator clauses!

Scaling Factor (Bart et al., 2016)

- For each instantiation, exactly one parameter variable will be assigned to true
- Then, we can keep for each CPT R one parameter variable θ_R **implicit**

Scaling Factor (Bart et al., 2016)

- For each instantiation, exactly one parameter variable will be assigned to true
- Then, we can keep for each CPT R one parameter variable θ_R **implicit**

A	B	C	$\mathbb{P}(C A,B)$
0	0	0	0.2
0	0	1	0.3
0	1	0	0.3
0	1	1	0.2
1	0	0	0
1	0	1	0
1	1	0	0.6
1	1	1	0.4

Scaling Factor (Bart et al., 2016)

- For each instantiation, exactly one parameter variable will be assigned to true
- Then, we can keep for each CPT R one parameter variable θ_R **implicit**

A	B	C	$\mathbb{P}(C A,B)$
0	0	0	0.2
0	0	1	0.3
0	1	0	0.3
0	1	1	0.2
1	0	0	0
1	0	1	0
1	1	0	0.6
1	1	1	0.4

Scaling Factor (Bart et al., 2016)

- For each instantiation, exactly one parameter variable will be assigned to true
- Then, we can keep for each CPT R one parameter variable θ_R **implicit**
- Instantiations with default value are left implicit

A	B	C	$\mathbb{P}(C A,B)$	0.3
0	0	0	0.2	
0	0	1	0.3	
0	1	0	0.3	
0	1	1	0.2	
1	0	0	0	
1	0	1	0	
1	1	0	0.6	
1	1	1	0.4	

Scaling Factor (Bart et al., 2016)

- For each instantiation, exactly one parameter variable will be assigned to true
- Then, we can keep for each CPT R one parameter variable θ_R **implicit**
- Instantiations with default value are left implicit

A	B	C	$\mathbb{P}(C A,B)$	0.3
0	0	0	0.2	0.2
0	0	1	0.3	0.3
0	1	0	0.3	
0	1	1	0.2	
1	0	0	0	
1	0	1	0	
1	1	0	0.6	
1	1	1	0.4	

Scaling Factor (Bart et al., 2016)

- For each instantiation, exactly one parameter variable will be assigned to true
- Then, we can keep for each CPT R one parameter variable θ_R **implicit**
- Instantiations with default value are left implicit

A	B	C	$\mathbb{P}(C A,B)$	
0	0	0	0.2	0.3
0	0	1	0.3	$\frac{0.2}{0.3}$
0	1	0	0.3	
0	1	1	0.2	$\frac{0.2}{0.3}$
1	0	0	0	
1	0	1	0	
1	1	0	0.6	$\frac{0.6}{0.3}$
1	1	1	0.4	$\frac{0.4}{0.3}$

Scaling Factor (Bart et al., 2016)

- For each instantiation, exactly one parameter variable will be assigned to true
- Then, we can keep for each CPT R one parameter variable θ_R **implicit**
- Instantiations with default value are left implicit

	A	B	C	$\mathbb{P}(C A,B)$	
θ_{c1}	0	0	0	0.2	0.3
	0	0	1	0.3	$\frac{0.2}{0.3} = w(\theta_{c1})$
	0	1	0	0.3	
θ_{c1}	0	1	1	0.2	$\frac{0.2}{0.3} = w(\theta_{c1})$
	1	0	0	0	
	1	0	1	0	
θ_{c2}	1	1	0	0.6	$\frac{0.6}{0.3} = w(\theta_{c2})$
θ_{c3}	1	1	1	0.4	$\frac{0.4}{0.3} = w(\theta_{c3})$

$$a_0 \wedge b_0 \wedge c_0 \rightarrow \theta_{c1}$$

$$a_0 \wedge b_1 \wedge c_1 \rightarrow \theta_{c1}$$

$$a_1 \wedge b_1 \wedge c_0 \rightarrow \theta_{c2}$$

$$a_1 \wedge b_1 \wedge c_1 \rightarrow \theta_{c3}$$

Scaling Factor (Bart et al., 2016)

- For each instantiation, exactly one parameter variable will be assigned to true
- Then, we can keep for each CPT R one parameter variable θ_R **implicit**
- Instantiations with default value are left implicit

	A	B	C	$\mathbb{P}(C A,B)$	
θ_{c1}	0	0	0	0.2	0.3
	0	0	1	0.3	$\frac{0.2}{0.3} = w(\theta_{c1})$
	0	1	0	0.3	
θ_{c1}	0	1	1	0.2	$\frac{0.2}{0.3} = w(\theta_{c1})$
	1	0	0	0	
	1	0	1	0	
θ_{c2}	1	1	0	0.6	$\frac{0.6}{0.3} = w(\theta_{c2})$
θ_{c3}	1	1	1	0.4	$\frac{0.4}{0.3} = w(\theta_{c3})$

$$a_0 \wedge b_0 \wedge c_0 \rightarrow \theta_{c1} \quad a_0 \wedge b_1 \wedge c_1 \rightarrow \theta_{c1}$$

$$a_1 \wedge b_1 \wedge c_0 \rightarrow \theta_{c2} \quad a_1 \wedge b_1 \wedge c_1 \rightarrow \theta_{c3}$$

- To make the translation faithful we now assign a specific weight to the negative parameter literals: $w(\neg\theta_{ij}) = 1 - w(\theta_{ij})$

Accounts for the instantiations with default implicit (0.3) and the factor for the remaining ones

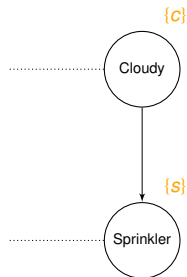
Running Example

C	$\mathbb{P}(C)$
0	0.5
1	0.5

S	C	$\mathbb{P}(S C)$
0	0	0.5
1	0	0.5
0	1	0.9
1	1	0.1

θ_{s3}

θ_{s4}



no need!

no need!

Running Example

$$w_0 = w_{s0} \times w_{c0}$$

no need!

C	$\mathbb{P}(C)$
0	0.5
1	0.5

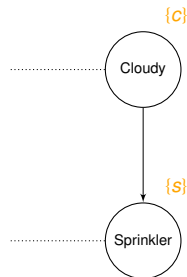
$$\bar{s} \wedge c \rightarrow \theta_{s3}$$

$$s \wedge c \rightarrow \theta_{s4}$$

S	C	$\mathbb{P}(S C)$
0	0	0.5
1	0	0.5
0	1	0.9
1	1	0.1

θ_{s3}

θ_{s4}



no need!

no need!

Running Example

$$w_0 = w_{s0} \times w_{c0}$$

no need!

C	P(C)
0	0.5
1	0.5

{c}



no need!

{s}



no need!

$$\bar{s} \wedge c \rightarrow \theta_{s3}$$

$$s \wedge c \rightarrow \theta_{s4}$$

S	C	P(S C)
0	0	0.5
1	0	0.5
0	1	0.9
1	1	0.1

$$w(\theta_{s3}) = 0.9/0.5$$

$$w(\theta_{s4}) = 0.1/0.5$$

- The weighted model count of (F, w, w_0) becomes:

$$W(F) = w_0 \left(\sum_{\mathbf{v} \in \{0,1\}^n, \mathbf{v} \models F} W(\mathbf{v}) \right)$$

Resources for SAT Modeling

For deeper insights into modeling problems using propositional logic and evaluating modern solvers, consider the following standard resources:

Satisfiability (SAT)

- **Handbook of Satisfiability** Biere et al. (2021)
The definitive guide to SAT theory, encoding techniques, and applications.
- **The SAT Competition**
Annual evaluation of state-of-the-art solvers.
<https://satcompetition.github.io/>

Model Counting (#SAT)

- **Model Counting Competition (MC)**
The premier benchmark track for exact and approximate #SAT solvers.
<https://mccompetition.org/>

Outline

- 1 How to model using SAT
- 2 Knowledge Compilation**
- 3 d-DNNF Queries
- 4 Conclusion and References